Sovereign AI Access Gateway: A Real-Time Streaming Data Backend for AI Governance, Security, and Operational Analytics

DLMDSEDE02 Data Engineering · Portfolio · Phase 1: Conception (Task 2)

Name: [Surname, First name] · Matriculation number: [00000000] · Date: [DD.MM.YYYY]

Concept

Problem and objective: European organisations increasingly use AI systems and autonomous agents but must keep sensitive data away from unapproved models, endpoints and jurisdictions, and prove that policies were enforced. This project builds a real-time streaming data backend that ingests AI-governance events, processes and aggregates them, and delivers live security and operational metrics.

Architecture: A FastAPI gateway authenticates callers, isolates tenants, detects sensitive data and prompt injection, and enforces policy-as-code. Each decision is stored with an event in a transactional outbox; a separate publisher delivers it to Apache Kafka, chosen for durable, partitioned, replayable buffering. Spark Structured Streaming, chosen for event-time windows, watermarks and checkpointed recovery, validates and enriches events and computes one-minute tumbling and five-minute sliding aggregations. PostgreSQL stores the results; a reporting API serves them to a React dashboard.

Data and reliability: A producer generates over 1,000,000 timestamped events. At-least-once delivery, deduplication, idempotent writes, checkpointing, retries and a dead-letter topic provide reliability; partitioning supports scaling.

Procedure and tools: Gateway first, then ingestion, processing and delivery, each verified by tests. Isolated Docker microservices, Docker Compose, versioned migrations and Git keep it reproducible and maintainable.

Security and governance: JWT, role-based access control, tenant isolation, least-privilege accounts, data minimisation, retention controls and tamper-evident audit records.

Advantages and disadvantages: Real-time, auditable visibility and reproducible deployment, at the cost of operational complexity and tuning windows, thresholds and resources.

Architecture draft

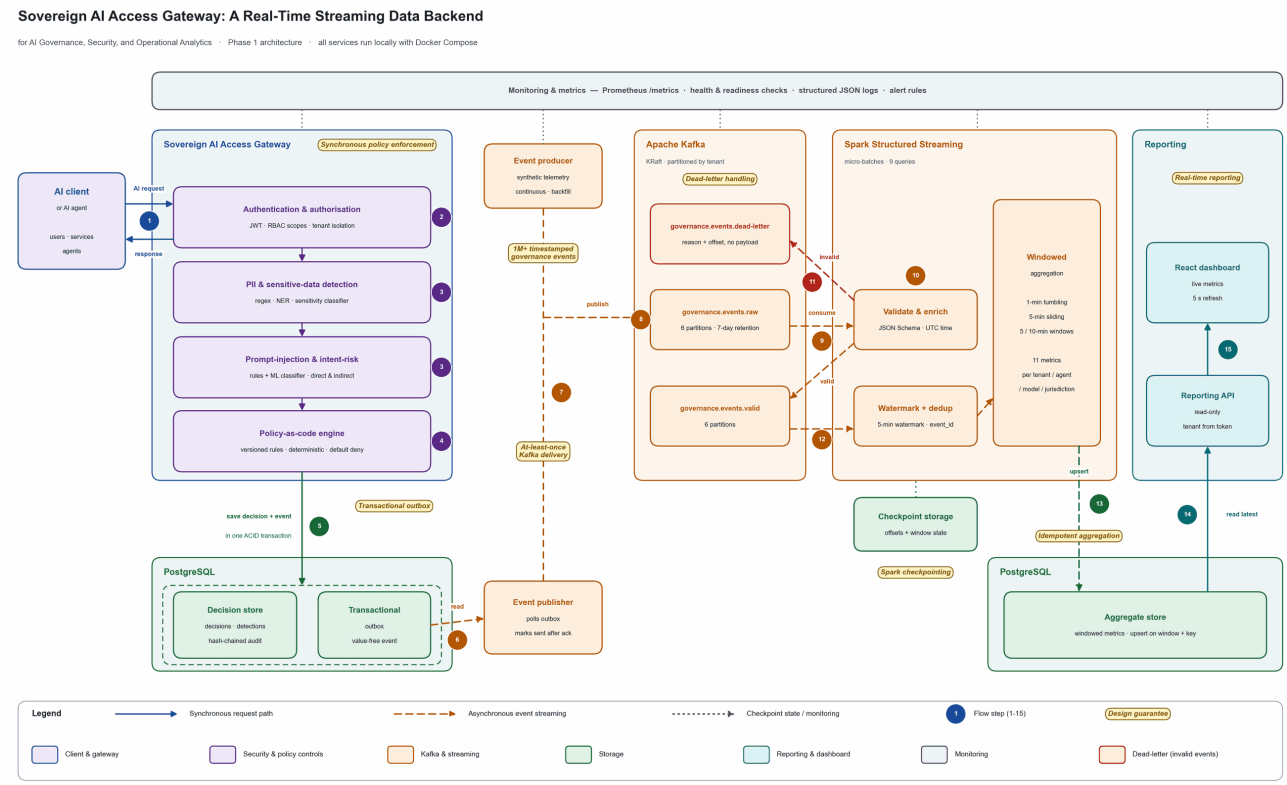


Figure 1: Architecture of the real-time streaming data backend (solid: synchronous request path; dashed: asynchronous event streaming).

Explanatory remarks

The architecture separates synchronous AI-security enforcement from asynchronous real-time analytics.

The gateway authenticates requests, detects sensitive data and prompt-injection risk, evaluates policy-as-code rules, and stores each decision with an outbox event in one transaction. A separate publisher delivers events to Kafka.

Kafka buffers and distributes governance events. Spark Structured Streaming validates, enriches, and aggregates the stream using one-minute tumbling and five-minute sliding windows. Aggregated metrics are stored in PostgreSQL and delivered through a real-time reporting API to the React dashboard.

Invalid events are routed to a dead-letter topic. Checkpointing enables recovery after processor failure. Tenant isolation, policy-as-code enforcement, data minimisation, and audit records support security and governance.